Docker processes our Dockerfile one instruction at a time. The first instruction—FROM—is treated differently from all the others. Docker checks whether we already have the specified image on our local system. If the image isn't available, Docker downloads the image, just like it did in Running a Ruby Script Without Ruby Installed, on page 7, when we ran docker run with the ruby:2.6 image for the first time.

All other instructions in the Dockerfile are processed in essentially the same way. Docker starts a throwaway container based on the image created in the previous step, and it executes the current Dockerfile instruction inside it. Then Docker *commits* the changes just made as a result of executing the instruction, creating a new *intermediate image* for this step.

You can see all this happening in the output, which is split into sections: one for each instruction in the Dockerfile. The output for each step follows a very regular format:

```
Line 1 Step <current step>/<total steps> : <Dockerfile instruction>
   2 ---> Running in <container ID>
   3 [Any output from running the instruction]
   4 Removing intermediate container <container ID>
   5 ---> <image ID>
```

For context, we are given the Dockerfile instruction, along with which step this is in the build process (line 1). Next we see the ID of the throwaway container used to execute the current Dockerfile instruction (line 2) followed by any output from running the instruction (line 3).

Every image created—even intermediate ones—are given a unique, randomly generated image ID. This is how Docker identifies images internally, and it gives us a way to refer to them. The ID of the image created in this step is shown on line 5.

Finally, on line 4, Docker tells us that it is deleting the throwaway container it used in this step.

When building a custom image, it's really the final image we're interested in. This represents the final state after all the instructions in the Dockerfile have been executed. That's why the output concludes by giving us the final image ID (if you're following along, your image ID will differ):

```
Successfully built a1df0eddba18
```

That all seems fair enough, but where *is* the image we just built?

The docker build command doesn't output a file; it simply makes the new image available, adding it to the list of images that Docker knows about. Docker